Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Иудин Е.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 18.10.26

Москва, 2026

Постановка задачи

Вариант 8.

Есть K массивов одинаковой длины. Необходимо сложить эти массивы. Необходимо предусмотреть стратегию, адаптирующуюся под количество массивов и их длину (по количеству операций)

Общий метод и алгоритм решения

Использованные системные вызовы:

- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg); – создаёт новый поток, который начинает выполнять функцию start_routine(arg).
- int pthread_join(pthread_t thread, void **retval); – блокирует вызывающий поток до завершения указанного потока. Гарантирует, что результаты работы потока видны после возврата.

Главный поток создаёт рабочие потоки (fork) вызовом pthread_create, каждый рабочий поток обрабатывает свою часть данных, после чего главный поток ждёт их завершения вызовом pthread_join (join) и проверяет результат. Максимальное число одновременно работающих потоков задаётся ключом запуска -t; главный поток вычислений не выполняет, а только создаёт рабочие потоки и ждёт их.

Сложение K (ключ -k) массивов длины N (ключ -n): результат R[i] = A[0][i] + A[1][i] + … + A[K-1][i]. Общее число операций сложения равно (K-1)·N. Реализованы две стратегии. Стратегия seq: один поток последовательно копирует первый массив в результат (memcpy) и прибавляет к нему остальные. Стратегия elem: диапазон индексов [0, N) делится на равные отрезки по числу потоков, каждый поток для своего отрезка проходит по всем K массивам и записывает сумму в соответствующую часть массива результата. Отрезки не пересекаются, поэтому потоки не пишут в общие ячейки, и мьютексы не нужны.

Адаптация под количество массивов и их длину выполняется по количеству операций (режим auto, ключ -s). Если число операций (K-1)·N меньше 50000, или указан один поток, программа считает в одном потоке. Иначе число потоков ограничивается так, чтобы на один поток приходилось не менее 4096 элементов: потоков = min(T, N / 4096). Если массивы слишком короткие (N / 4096 < 2), также выбирается один поток. Стратегию можно задать вручную значениями seq и elem.

Массив с номером k (k = 0…K-1) заполняется по формуле A[k][i] = i + k, поэтому правильную сумму в позиции i можно вычислить заранее: K·i + K·(K-1)/2. После вычислений программа сверяет результат с этой формулой и выводит check=OK или check=FAIL. Для замера время вычисления измеряется несколько раз (ключ -r), берётся минимальное; генерация данных и проверка в замер не входят. По времени считаются ускорение S = Ts / Tp (Ts – время последовательной реализации, Tp – многопоточная реализация, p – количество потоков) и эффективность X = S / p. Для демонстрации числа потоков средствами ОС используется ключ -w: рабочие потоки не завершаются заданное число секунд.

Код программы

sum_arrays.c

```
// compile:
// gcc -pthread -o sum_arrays sum_arrays.c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

#define SEQ_THRESHOLD 50000LL
#define MIN_CHUNK 4096

typedef long long ll;

typedef enum {
    S_AUTO,
    S_SEQ,
    S_ELEM
} strategy_mode;

typedef struct {
    ll from, to;
} range_t;

static int K = 8;
static ll N = 1000000;
static int T = 1;
static ll **A;
static ll *res;
static int hold_sec = 0;
```

```
static void hold_threads(void) {
    if (hold_sec > 0){
        sleep(hold_sec);
    }
}

static double now(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

static void run_seq(void) {
    memcpy(res, A[0], N * sizeof(ll));
    for (int k = 1; k < K; k++) {
        const ll *a = A[k];
        for (ll i = 0; i < N; i++) {
            res[i] += a[i];
        }
    }
}

static void *worker_elem(void *p) {
    range_t *r = p;
    memcpy(res + r->from, A[0] + r->from, (r->to - r->from) * sizeof(ll));
    for (int k = 1; k < K; k++) {
        const ll *a = A[k];
        for (ll i = r->from; i < r->to; i++) {
            res[i] += a[i];
        }
    }
    hold_threads();
    return NULL;
```

```
}

static void run_elem(int threads) {
    pthread_t th[threads];
    range_t rg[threads];
    ll chunk = (N + threads - 1) / threads;
    int started = 0;
    for (int t = 0; t < threads; t++) {
        rg[t].from = (ll)t * chunk;
        rg[t].to = rg[t].from + chunk;
        if (rg[t].from >= N) break;
        if (rg[t].to > N) rg[t].to = N;
        pthread_create(&th[t], NULL, worker_elem, &rg[t]);
        started++;
    }
    for (int t = 0; t < started; t++) {
        pthread_join(th[t], NULL);
    }
}

static const char *choose(strategy_mode m, int *threads) {
    if (m == S_SEQ || T <= 1) { *threads = 1; return "seq"; }
    if (m == S_ELEM) return "elem";
    ll ops = (ll)(K - 1) * N;
    if (ops < SEQ_THRESHOLD) { *threads = 1; return "seq"; }
    ll max_by_n = N / MIN_CHUNK;
    if (max_by_n < 2) { *threads = 1; return "seq"; }
    if (max_by_n < T) *threads = (int)max_by_n;
    return "elem";
}

int main(int argc, char **argv) {
    strategy_mode mode = S_AUTO;
```

```
int reps = 5, hold = 0, opt;
while ((opt = getopt(argc, argv, "k:n:t:s:r:w:")) != -1) {
   switch (opt) {
   case 'k': K = atoi(optarg); break;
   case 'n': N = atoll(optarg); break;
   case 't': T = atoi(optarg); break;
   case 'r': reps = atoi(optarg); break;
   case 'w': hold = atoi(optarg); break;
   case 's':
      if (!strcmp(optarg, "auto")) mode = S_AUTO;
      else if (!strcmp(optarg, "seq")) mode = S_SEQ;
      else if (!strcmp(optarg, "elem")) mode = S_ELEM;
      else { fprintf(stderr, "bad -s\n"); return 1; }
      break;
   default:
      fprintf(stderr, "usage: %s -k K -n N -t T [-s auto|seq|elem] [-r R] [-w SEC]\n", argv[0]);
      return 1;
   }
}
if (K < 1 || N < 1 || T < 1) { fprintf(stderr, "bad params\n"); return 1; }

A = malloc(K * sizeof(ll*));
for (int k = 0; k < K; k++) {
   A[k] = malloc(N * sizeof(ll));
   for (ll i = 0; i < N; i++) {
      A[k][i] = i + k;
   }
}
res = malloc(N * sizeof(ll));

hold_sec = hold;
if (hold > 0) reps = 1;
int th = T;
```

```
    const char *strat = choose(mode, &th);
    double best = 1e30;
    for (int r = 0; r < reps; r++) {
        double t0 = now();
        if (!strcmp(strat, "seq")) {
            run_seq();
        }
        else {
            run_elem(th);
        }
        double dt = now() - t0;
        if (dt < best) {
            best = dt;
        }
    }

    int ok = 1;
    for (ll i = 0; i < N && ok; i++) {
        ll s = 0;
        s = K * i + K * (K - 1) / 2;
        if (s != res[i]) {
            ok = 0;
            fprintf(stderr, "mistake at %lld\n", i);
        }
    }

    printf("K=%d N=%lld T=%d strategy=%s used_threads=%d time=%.6f check=%s\n",
           K, N, T, strat, th, best * 100.f, ok ? "OK" : "FAIL");

    return 0;
}
```

Протокол работы программы

```
4oddy@MacBook-Pro-Egor-3 lab2 % ./sum_arrays -k 1000 -n 100000 -r 5 -t 1024
K=1000 N=100000 T=1024 strategy=elem used_threads=24 time=1.239100 check=OK
4oddy@MacBook-Pro-Egor-3 lab2 %
```

Вывод

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	113.526	1.00	1.00
2	61.7	1.84	0.92
4	33.6	3.38	0.84
8	19.03	5.97	0.75
14	14.26	7.96	0.57
64	12.99	8.74	0.14
128	13.21	8.59	0.067
1024	13.46	8.43	0.008

Тестирование проводилось на MacBook Pro с процессором Apple M4 Pro (14 ядер: 10 производительных и 4 энергоэффективных) и 24 ГБ унифицированной памяти при K = 100 массивов длиной N = 10000000 (суммарный объём входных данных около 8 ГБ).

При 2, 4 и 8 потоках ускорение растёт почти линейно (1.84, 3.38 и 5.97), эффективность снижается плавно (0.92, 0.84, 0.75). Постепенное падение эффективности объясняется тем, что потоки начинают конкурировать за общий ресурс – память.

При 14 потоках (по числу ядер) ускорение составляет всего 7.96, эффективность падает до 0.57, то есть рост заметно замедляется. Можно предположить, что задача упирается в пропускную способность памяти, а не в вычисления: на одно сложение приходится чтение 8 байт, и когда потоки одновременно читают данные, они делят общую шину памяти, поэтому добавление потоков перестаёт давать пропорциональный выигрыш. К тому же, на моем процессоре 14 ядер неодинаковы: 10 производительных и 4 энергоэффективных, которые заметно медленнее. Отрезки при 14 потоках разделены поровну, поэтому потоки на производительных ядрах заканчивают раньше и ждут в pthread_join потоки, оказавшиеся на энергоэффективных ядрах: время определяется самым медленным потоком.

Дальнейшее увеличение числа потоков до 64 всё ещё немного уменьшает время (12.99 против 14.26 при 14 потоках, ускорение 8.74) хотя потоков уже больше, чем ядер. При большом числе потоков каждому достаётся меньший отрезок, операционная система распределяет их по ядрам по мере освобождения, производительные ядра успевают обработать больше потоков, чем энергоэффективные, и нагрузка выравнивается, соответственно ускорение еще больше.

При 128 и 1024 потоках время перестаёт уменьшаться и даже немного растёт (13.21 и 13.46), ускорение остаётся около 8.4–8.7. Лишние потоки не добавляют никакой производительности, а только увеличивают накладные расходы на создание потоков, переключение контекста и выделение

стеков. Поэтому эффективность падает примерно обратно пропорционально числу потоков (0.07 при 128 и 0.008 при 1024).

Таким образом, минимальное время достигается при большом числе потоков (около 64), но наиболее эффективно используются ресурсы при небольшом числе потоков (2–4, эффективность выше 0.8). Разумным компромиссом является число потоков, близкое к числу ядер (8–14): ускорение 6–8 при эффективности 0.57–0.75.

В заключение хочу отметить, что в ходе выполнения лабораторной работы, я усвоил новую тему под названием многопоточность. Была реализована многопоточная программа, которая суммирует массивы в один. Также «эмпирическим» путем было выявлено самое оптимальное число потоков в программах, где необходимо увеличить производительность. Были использованы основные метрики, а также проведено сравнение последовательной программы и многопоточной.